

Divide and Conquer ve Verimli Sıralama

Hafta 3

1. <https://gemini.google.com/share/df9c2f6b0d49>

Böl ve Yönet (Divide and Conquer) Paradigması

- D&C'nin üç aşaması: Böl, Yönet ve Birleştir.
- Karmaşık problemleri daha küçük alt problemlere indirger.
- Alt problemler ana problemle aynı yapıdadır.
- Master Teoremi ile karmaşıklık analizi yapılır.

<https://gemini.google.com/share/df9c2f6b0d49>

Merge Sort: Birleştirme Mantığı

- İki sıralı diziyi tek bir sıralı dizi yapar.
- $O(n)$ zamanında çalışan lineer bir operasyondur.
- Ekstra bellek ($O(n)$) gerektiren bir birleştirme sürecidir.
- İki deste kartın birleştirilmesi analogisiyle açıklanır.

<https://gemini.google.com/share/df9c2f6b0d49>

Algorithm

```
1  procedure merge(A, p, q, r):
2      n1 = q - p + 1
3      n2 = r - q
4      let L[1..n1+1] and R[1..n2+1] be new arrays
5      for i = 1 to n1: L[i] = A[p + i - 1]
6      for j = 1 to n2: R[j] = A[q + j]
7      L[n1+1] = infinity, R[n2+1] = infinity
8      i = 1, j = 1
9      for k = p to r:
10         if L[i] <= R[j]: A[k] = L[i]; i = i + 1
11         else: A[k] = R[j]; j = j + 1
```

gemini anlatiklari bu kod hakkında doc.

<https://gemini.google.com/share/df9c2f6b0d49>

Adım 1: Hazırlık ve Boyut Hesaplama

$A = [\underbrace{3, 9, 10, 27, 38}_{\text{Sol (Sıralı)}}, \underbrace{8, 13, 21, 25, 50}_{\text{Sağ (Sıralı)}}]$ dizisini birleştirelim.

- **n1 (Sol Parça):** $q - p + 1 \Rightarrow 5 - 1 + 1 = 5$
- **n2 (Sağ Parça):** $r - q \Rightarrow 10 - 5 = 5$
- **Bellek:** L ve R dizileri $5 + 1$ (sentinel için) boyutunda oluşturulur.

Durum:

L = [?, ?, ?, ?, ?, ∞]

R = [?, ?, ?, ?, ?, ∞]

<https://gemini.google.com/share/df9c2f6b0d49>

Adım 2: Elemanların Kopyalanması

Sıralı alt diziler ana diziden geçici belleklere (L ve R) taşınır.

L (Left)

L = [3, 9, 10, 27, 38, ∞]

A[1..5] aralığı kopyalandı

R (Right)

R = [8, 13, 21, 25, 50, ∞]

A[6..10] aralığı kopyalandı

Neden Sonsuz (∞)?

Nöbetçi değer (sentinel), dizilerden biri tamamen tükendiğinde karşılaştırmanın hata vermeden devam etmesini sağlar.

<https://gemini.google.com/share/df9c2f6b0d49>

Adım 3: Karşılaştırma ve Birleştirme

k indisi ana dizide ilerlerken, i L ve j R indisleri en küçük elemanı seçer.

k (Ana Dizi)	Karşılaştırma	İşlem	A1...k Durumu
1	<code>L[1](3) <= R[1](8)</code>	<code>A[1]=3, i++</code>	<code>[3, ...]</code>
2	<code>L[2](9) <= R[1](8)</code>	<code>A[2]=8, j++</code>	<code>[3, 8, ...]</code>
3	<code>L[2](9) <= R[2](13)</code>	<code>A[3]=9, i++</code>	<code>[3, 8, 9, ...]</code>
4	<code>L[3](10) <= R[2](13)</code>	<code>A[4]=10, i++</code>	<code>[3, 8, 9, 10, ...]</code>
5	<code>L[4](27) <= R[2](13)</code>	<code>A[5]=13, j++</code>	<code>[3, 8, 9, 10, 13, ...]</code>

<https://gemini.google.com/share/df9c2f6b0d49>

Adım 4: Birleştirmenin Tamamlanması

Tüm elemanlar $O(n)$ sürede ana diziye yerleştirildi.

$A = [3, 8, 9, 10, 13, 21, 25, 27, 38, 50]$

Zaman Karmaşıklığı

Sadece bir tur: $O(n)$

Kararlılık (Stability)

Eşit değerlerde $L[i]$ seçildiği için
Stable'dir.

Bellek Kullanımı

Ekstra $O(n)$ yer gerektirir.

<https://gemini.google.com/share/df9c2f6b0d49>

Özet: "Divide and Conquer"

Merge Sort'un kalbi bu birleştirme (`merge`) fonksiyonudur.

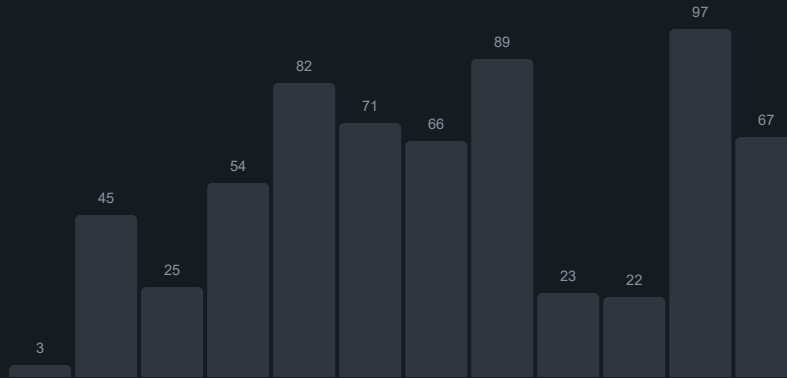
Görselleştirme için dizinin sonuna eklenen ∞ değerlerinin mantığını unutmayın!

Merge Sort

Time Complexity: $O(n \log n)$

Diziyi her adımda ikiye böler ve tek eleman kalana kadar iner. Ardından sıralı bir şekilde birleştirerek yukarı çıkar.

Ready



Comparisons

0

Recursion Depth

0

<https://gemini.google.com/share/df9c2f6b0d49>

merge_sort.py

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    # Compare and merge elements...
```



Play



Speed



Elements



Merge Sort: Divide, Conquer, Combine

- Dizi sürekli ikiye bölünerek tek elemana indirilir.
- Tek elemanlı diziler her zaman sıralı kabul edilir.
- Birleştirme adımı aşağıdan yukarıya doğru sıralamayı inşa eder.
- Her zaman kararlı (stable) bir performans sergiler.

<https://gemini.google.com/share/df9c2f6b0d49>

Algorithm

```
1  procedure merge_sort(A, p, r):  
2      if p < r:  
3          q = (p + r) / 2  
4          merge_sort(A, p, q)  
5          merge_sort(A, q + 1, r)  
6          merge(A, p, q, r)
```

gemini anlatiklari bu kod hakkında doc.

<https://gemini.google.com/share/df9c2f6b0d49>

Merge Sort: Zaman Karmaşıklığı Analizi

- Rekürans bağıntısı: $T(n) = 2T(n/2) + n$.
- Tüm durumlarda (En iyi/kötü) $O(n \log n)$ çalışır.
- Logaritmik çarpan, karesel sınıra göre devasa hız sağlar.
- Verinin başlangıç dizilimi performansı kesinlikle etkilemez.

<https://gemini.google.com/share/df9c2f6b0d49>

Merge Sort: Bellek Karmaşıklığı (Space Complexity)

- In-place çalışmaz; $O(n)$ yardımcı belleğe ihtiyaç duyar.
- Büyük verilerde RAM kısıtı önemli bir darboğazdır.
- Kararlı (stable) olması en büyük avantajlarından biridir.
- Bellek ve hız arasındaki klasik mühendislik takasıdır.

<https://gemini.google.com/share/df9c2f6b0d49>

Quick Sort: Pivot Seçimi ve Partitioning

- Bir pivot eleman seçilerek dizi ikiye bölünür.
- Küçükler solda, büyükler sağda toplanarak düzenlenir.
- Pivotun yeri işlem sonunda kalıcı olarak sabitlenir.
- Performansı doğrudan pivotun seçilme kalitesine bağlıdır.

<https://gemini.google.com/share/89fe23c6b536>

Quick Sort

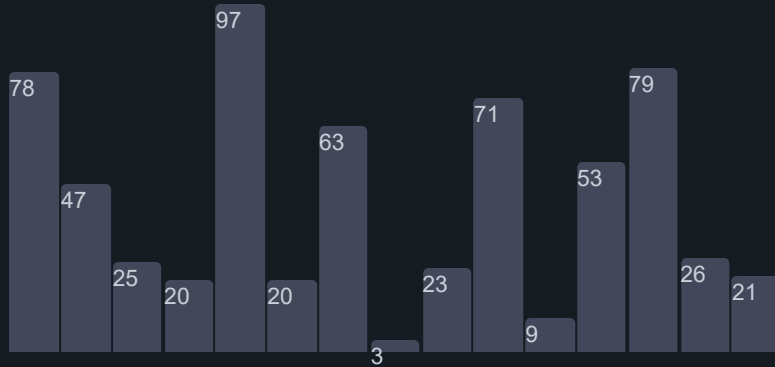
Time Complexity: $O(n \log n)$

Pivot elemanını seçer ve diğer elemanları pivotun değerine göre sağa veya sola ayırır (Partitioning).

Ready

<https://gemini.google.com/share/89fe23c6b536>

quick_sort.py



Swaps

0

Comparisons

0

```
def quick_sort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
```



▶ Play



Speed

Elements

Quick Sort: Lomuto Partitioning Şeması

- Tek bir tarama indisiyle çalışan basit şema.
- Son eleman pivot olarak seçilir ve karşılaştırılır.
- Yazılması ve anlaşılması en kolay bölümlleme yöntemidir.
- Hoare şemasına göre daha fazla swap işlemi yapar.

<https://gemini.google.com/share/89fe23c6b536>

Algorithm

```
1  procedure partition(A, p, r):  
2      pivot = A[r]  
3      i = p - 1  
4      for j = p to r - 1:  
5          if A[j] <= pivot:  
6              i = i + 1  
7              swap(A[i], A[j])  
8      swap(A[i + 1], A[r])  
9      return i + 1
```

<https://gemini.google.com/share/89fe23c6b536>

Lomuto Bölümleme Şeması: Başlangıç (1/3)

Senaryo: Dizinin son elemanını **Pivot** seçerek, küçükleri sola, büyükleri sağa toplayacağız.

İndeks	0	1	2	3	4	5	6	7	8	9 (P)
Değer	15	42	7	12	55	20	60	5	30	25

****Pivot (\$P\$):**** 25

****Sınır Hattı (\$i\$):**** -1 (Henüz küçük eleman bulunamadı)

****Kaşif (\$j\$):**** 0'dan başlayarak 8'e kadar tarayacak.

****Kural:****

Eğer ``Dizi[j] <= Pivot`` ise:

1. `$i$` indeksini 1 artır.
2. ``Dizi[i]`` ile ``Dizi[j]`` elemanlarını ****takas et****.

<https://gemini.google.com/share/89fe23c6b536>

Lomuto: Adım Adım Tarama (2/3)

- **j=0 (15):** $15 \leq 25$ mi? **Evet.** i sıfır olur. 15 kendisiyle takas edilir. 15, 42, 7, 12, 55, 20, 60, 5, 30 | 25
- **j=1 (42):** $42 \leq 25$ mi? **Hayır.** i sabit kalır (0). Sadece j ilerler.
- **j=2 (7):** $7 \leq 25$ mi? **Evet.** i bir artar (1). 42 ile 7 takas edilir. 15, 7, 42, 12, 55, 20, 60, 5, 30 | 25
- **j=3 (12):** $12 \leq 25$ mi? **Evet.** i bir artar (2). 42 ile 12 takas edilir. 15, 7, 12, 42, 55, 20, 60, 5, 30 | 25
- **j=5 (20):** ($j=4$ geçildi) $20 \leq 25$ mi? **Evet.** i bir artar (3). 42 ile 20 takas edilir. 15, 7, 12, 20, 55, 42, 60, 5, 30 | 25
- **j=7 (5):** $5 \leq 25$ mi? **Evet.** i bir artar (4). 55 ile 5 takas edilir. 15, 7, 12, 20, 5, 42, 60, 55, 30 | 25

Lomuto: Pivotun Merkeze Alınması (3/3)

Tarama bitti ($j=8$). i indisi şu an 4. konumda (Değer: 5).

Final Hamlesi: Pivotu ($i + 1$) konumuna yerleştirmeliyiz. Bu yüzden $i + 1$ (5. indeks) olan **42** ile Pivot (**25**) takas edilir.

15 7 12 20 5 25 60 55 30 42

****Sol Bölge (Küçükler):****

[15, 7, 12, 20, 5]

Hepsi 25'ten küçük.

****Sağ Bölge (Büyükler):****

[60, 55, 30, 42]

Hepsi 25'ten büyük.

Outcome: Pivot tam yerini buldu. Algoritma sağ ve sol için rekürsif devam eder.

Quick Sort: Hoare Partitioning Şeması

- İki uçtan merkeze doğru daralan tarama yapar.
- Lomuto'ya göre daha az swap yaparak hızlanır.
- Pivot etrafında daha dengeli bir bölümleme sağlar.
- Orijinal Quick Sort makalesindeki klasik bölümleme yöntemidir.

<https://gemini.google.com/share/89fe23c6b536>

Algorithm

```
1  procedure partition(A, p, r):
2      pivot = A[p]
3      i = p - 1
4      j = r + 1
5      while true:
6          do: j = j - 1 while A[j] > pivot
7          do: i = i + 1 while A[i] < pivot
8          if i < j: swap(A[i], A[j])
9          else: return j
```

<https://gemini.google.com/share/89fe23c6b536>

Hoare Bölümleme Şeması: Başlangıç (1/3)

Strateji: Çift yönlü tarama. Soldan büyük, sağdan küçük eleman bulup takas edeceğiz.

İndeks	0 (P)	1	2	3	4	5	6	7	8	9
Değer	25	42	7	12	55	20	60	5	30	15

****Pivot (\$P\$):**** 25 (İlk eleman seçildi)
****Sol İşaretçi (\$i\$):**** -1'den başlar, sağa gider.
****Sağ İşaretçi (\$j\$):**** 10'dan başlar, sola gider.

****Kural:****
1. \$i\$, `Dizi[i] >= Pivot` olana kadar sağa ilerler.
2. \$j\$, `Dizi[j] <= Pivot` olana kadar sola ilerler.
3. \$i < j\$ ise bu iki elemanı ****takas et****.

Hoare: Karşılıklı Kuşatma (2/3)

- **1. Tur:**
 - i ilerler: 42'de durur ($42 > 25$).
 - j ilerler: 15'te durur ($15 < 25$).
 - **Swap(42, 15):** 25, 15, 7, 12, 55, 20, 60, 5, 30, 42
- **2. Tur:**
 - i ilerler: 55'te durur ($55 > 25$).
 - j ilerler: 5'te durur ($5 < 25$).
 - **Swap(55, 5):** 25, 15, 7, 12, 5, 20, 60, 55, 30, 42
- **3. Tur:**
 - i ilerler: 60'ta durur ($60 > 25$).
 - j ilerler: 20'de durur ($20 < 25$).
 - **Swap(60, 20):** 25, 15, 7, 12, 5, 20, 60, 55, 30, 42

<https://gemini.google.com/share/89fe23c6b536>

****Dikkat:**** İşaretçiler birbirine çok yaklaştı. Bir sonraki adımda çakışma kontrol edilecek.

Hoare: İşaretçilerin Karşılaşması (3/3)

Son Tarama:

i sağa gider ve 60'ta durur.

j sola gider ve 20'de durur.

<https://gemini.google.com/share/89fe23c6b536>

Durum: j , i 'nin soluna geçti ($j < i$). Döngü kırılır ve bölümlleme biter.

25, 15, 7, 12, 5, 20

60, 55, 30, 42

Sol Bölge (Küçükler)

Sağ Bölge (Büyükler)

****Verimlilik:**** Lomuto'ya göre çok daha az takas (swap) yapıldı. Elemanlar "yanlış yerde" yakalandığı anda karşıya fırlatıldı.

****Önemli:**** Hoare şemasında pivot, Lomuto'daki gibi tam orta noktaya (final yerine) gelmeyebilir; sadece iki bölgeyi birbirinden ayırır.

Outcome: Hoare şeması, büyük veri setlerinde %300'e varan hız avantajı sağlar.

Quick Sort: En Kötü Durum (Worst Case) Analizi

- Kötü pivot seçimi performansı $O(n^2)$ seviyesine düşürür.
- Zaten sıralı dizide ilk/son eleman pivot seçilirse oluşur.
- Rekürans bağıntısı: $T(n) = T(n - 1) + n$.
- Dengesiz bölünme, logaritmik avantajı tamamen ortadan kaldırır.

<https://gemini.google.com/share/89fe23c6b536>

Quick Sort: Ortalama Durum (Average Case) Analizi

- Rastgele verilerde ortalama performans $O(n \log n)$ 'dir.
- Dengesiz bölünmeler (1:9 gibi) bile logaritmik kalır.
- Pratikte Merge Sort'tan daha hızlı olduğu gözlemlenir.
- Cache dostu yapısı sayesinde donanımla uyumlu çalışır.

<https://gemini.google.com/share/89fe23c6b536>

Randomized Quick Sort

- Pivot eleman her adımda rastgele seçilerek ilerlenir.
- Verinin sırasına olan bağımlılık tamamen ortadan kaldırılır.
- Worst-case ihtimali matematiksel olarak yok sayılabilir seviyededir.
- Endüstriyel sistemlerde standart güvenlik ve hız önlemidir.

<https://gemini.google.com/share/89fe23c6b536>

Algorithm

```
1  procedure randomized_partition(A, p, r):  
2      i = random(p, r)  
3      swap(A[r], A[i])  
4      return partition(A, p, r)
```

<https://visualgo.net/en/sorting>

Gemini deleted this part from the chat so you can review the A4 copybook that has a girl with yellow hair on it

Quick Sort: Stack Derinliği ve Tail Recursion

- Özyineleme derinliği Stack Overflow riski yaratır.
- En kötü durumda $O(n)$ olan derinlik tehlikelidir.
- Tail recursion optimizasyonu ile derinlik $O(\log n)$ olur.
- Önce küçük parçanın çözülmesi bellek tüketimini minimize eder.

<https://gemini.google.com/share/37222ac1ef35>

Algorithm

```
1  procedure quick_sort_optimized(A, p, r):  
2      while p < r:  
3          q = partition(A, p, r)  
4          if q - p < r - q:  
5              quick_sort_optimized(A, p, q - 1)  
6              p = q + 1  
7          else:  
8              quick_sort_optimized(A, q + 1, r)  
9              r = q - 1
```

<https://gemini.google.com/share/37222ac1ef35>

Merge Sort vs Quick Sort Karşılaştırması

- Merge Sort kararlıdır, Quick Sort ise kararsızdır.
- Merge Sort $O(n)$ bellek ister, Quick Sort yerindedir.
- Büyük veri setlerinde Quick Sort ham hızda kazanır.
- Linked List yapılarında Merge Sort daha verimli çalışır.

<https://gemini.google.com/share/4e7a8cb00e35>

Yığın (Heap) Veri Yapısına Giriş

- Tam bir ikili ağaç (complete binary tree) yapısıdır.
- Dizi içerisinde indis hesaplamalarıyla verimli saklanır.
- Max-Heap: Ebeveyn düğüm daima çocuklarından büyüktür.
- Öncelikli kuyruk (Priority Queue) yapılarının temelidir.

<https://gemini.google.com/share/4e7a8cb00e35>

Heapify Operasyonu

- Bozulan heap özelliğini tamir etme mekanizmasıdır.
- Yukarıdan aşağıya (sift-down) doğru elemanların yerini değiştirir.
- Zaman karmaşıklığı ağacın yüksekliğiyle orantılıdır: $O(\log n)$.
- Yığın yapısının bütünlüğünü koruyan en temel mekanizmadır.

<https://gemini.google.com/share/4e7a8cb00e35>

Algorithm

```
1  procedure max_heapify(A, i):
2      l = 2 * i
3      r = 2 * i + 1
4      if l <= heap_size and A[l] > A[i]: largest = l
5      else: largest = i
6      if r <= heap_size and A[r] > A[largest]: largest = r
7      if largest != i:
8          swap(A[i], A[largest])
9          max_heapify(A, largest)
```

<https://gemini.google.com/share/4e7a8cb00e35>

Heapify (Max-Heapify) Operasyonu (1/2)

Durum: Elimizde neredeyse tam bir Max-Heap var, ancak kökteki eleman (12) kuralı bozuyor. Onu aşağı süzerek (sift-down) heap yapısını tamir edeceğiz.

İndeks	1 (Kök)	2	3	4	5	6	7	8	9	10
Değer	12	85	70	50	40	65	60	10	20	35

****Hedef Eleman:**** 12 (İndeks 1)

****Çocuklar:****

- Sol Çocuk ($2i$): İndeks 2 (85)
- Sağ Çocuk ($2i+1$): İndeks 3 (70)

****Kural:****

1. Baba, sol ve sağ çocuk arasından ****en büyüğü**** seçilir.
2. Eğer en büyük çocuk babadan büyükse, ****takas (swap)**** yapılır.
3. İşlem yeni konumda tekrar eder.

Build-Heap: Yığın İnşa Etme

- Rastgele bir diziyi heap yapısına dönüştürme sürecidir.
- Yapraklardan köke doğru heapify operasyonları yapılarak ilerlenir.
- Toplam karmaşıklık şaşırtıcı bir şekilde lineerdir: $O(n)$.
- Sıralama öncesi veriyi hazırlayan verimli bir ön işlemedir.

<https://gemini.google.com/share/ea405f548f10>

Algorithm

```
1  procedure build_max_heap(A):  
2      heap_size = length(A)  
3      for i = floor(length(A) / 2) downto 1:  
4          max_heapify(A, i)
```

<https://gemini.google.com/share/ea405f548f10>

Build-Heap: Rastgele Diziden Max-Heap'e (1/2)

Girdi: 15, 20, 7, 45, 12, 60, 30, 10, 5, 25 ($n = 10$)

Kural: Yapraklar (çocuğu olmayanlar) zaten birer heap'tir. İşlem $n/2$ konumundan başlar.

İndeks	1	2	3	4	5 (Başlangıç)	6	7	8	9	10
Değer	15	20	7	45	12	60	30	10	5	25

****Neden 5. İndeks?***

$10 / 2 = 5$. indeks, yaprak olmayan ****son**** düğümdür.
6, 7, 8, 9 ve 10. indeksler yapraktır, işlem gerektirmez.

****İş Akışı:****

$\$i = 5$ 'den $\$i = 1$ 'e doğru geri sayarak her düğüm için ****Max-Heapify**** çağrılır.

<https://gemini.google.com/share/ea405f548f10>

Build-Heap: Geriye Doğru Heapify (2/2)

- **Adım 1** ($i = 5$): Değer **12**. Tek çocuğu (İndeks 10) **25**.

25 > 12 olduğu için takas edilir. -> [... , 25, 60, 30, 10, 5, 12]

- **Adım 2** ($i = 4$): Değer **45**. Çocukları 10 ve 5.

45 zaten çocuklarından büyük. **Değişim yok**.

<https://gemini.google.com/share/ea405f548f1>

- **Adım 3** ($i = 3$): Değer **7**. Çocukları **60** ve 30.

60 en büyük. Takas edilir. -> [15, 20, 60, 45, 25, 7, 30, 10, 5, 12]

- **Adım 4** ($i = 2$): Değer **20**. Çocukları **45** ve 25.

45 en büyük. Takas edilir. 20 aşağı süzülür (45'in eski yerine).

Yeni yerinde çocukları 10 ve 5'ten büyüktür. **Durur**.

- **Adım 5** ($i = 1$ - **KÖK**): Değer **15**. Çocukları 45 ve **60**.

60 en büyük. Takas edilir. 15 aşağı süzülür (60'ın eski yerine).

Yeni yerinde çocukları **30** ve 7 arasından 30 büyüktür. Tekrar takas edilir.

Final Max-Heap: [60, 45, 30, 20, 25, 15, 7, 10, 5, 12]

Heap Sort Algoritması

- Önce bir Max-Heap inşa edilir ($O(n)$).
- Kökten en büyük çekilir ve sona atılır.
- Kalan dizi tekrar heapify yapılır ($O(\log n)$).
- Bu süreç n kez tekrarlanarak sıralı dizi oluşturulur.

<https://gemini.google.com/share/ea405f548f10>

Algorithm

```
1  procedure heap_sort(A):  
2      build_max_heap(A)  
3      for i = length(A) downto 2:  
4          swap(A[1], A[i])  
5          heap_size = heap_size - 1  
6          max_heapify(A, 1)
```

<https://gemini.google.com/share/ea405f548f10>

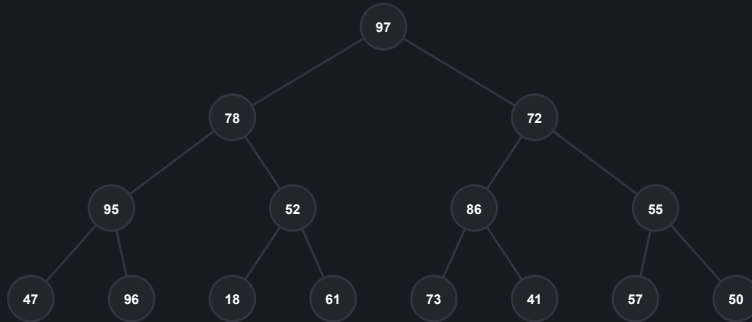
Heap Sort

Diziyi bir Max-Heap yapısına dönüştürür ve ardından kökteki en büyük elemanı sona taşıyarak sıralar.

Time Complexity: $O(n \log n)$

Array

Tree



```
heap_sort.py
def heap_sort(arr):
    build_max_heap(arr)
    for i in range(len(arr)-1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        max_heapify(arr, 0, i)
```



▶ Play



Heap Sort: Zaman ve Bellek Analizi

- Tüm durumlarda garantili $O(n \log n)$ performansı sunar.
- Yerde (In-place) çalışarak ekstra bellek ihtiyacını yok eder.
- Kararsız (Unstable) bir yapıya sahip olduğu unutulmamalıdır.
- Quick Sort'un en kötü durum riskine karşı güvenli bir limandır.

Sıralama Alt Sınırı ($\Omega(n \log n)$) İspatı

- Karşılaştırma tabanlı algoritmalar $n \log n$ 'den hızlı olamaz.
- Karar Ağacı (Decision Tree) modeliyle matematiksel ispatı yapılır.
- $n!$ yapraklı bir ağacın yüksekliği $\log(n!) = n \log n$ 'dir.
- Bu limit, algoritmaların ulaşabileceği nihai fiziksel sınırdır.

<https://gemini.google.com/share/8079af9dd156>

Merge Sort Pseudocode İncelemesi

- 'Divide' adımı diziyi sürekli ikiye bölerek ilerler.
- 'Merge' fonksiyonu asıl sıralama ve birleştirme işini yapar.
- Özyineleme, alt problemler çözülene kadar beklemeyi sağlar.
- Parametreler (p, q, r) dizideki sınırları hassas belirler.

Algorithm

```
1  procedure merge_sort(A, p, r):  
2      if p < r:  
3          q = floor((p + r) / 2)  
4          merge_sort(A, p, q)  
5          merge_sort(A, q + 1, r)  
6          merge(A, p, q, r)
```

Quick Sort Pseudocode İncelemesi

- 'Partition' fonksiyonu pivotu yerine yerleştirip döndürür.
- Pivotun solunda kalanlar küçük, sağında kalanlar büyük olur.
- Özyineleme, pivot hariç her iki yana ayrı uygulanır.
- Kodun performansı tamamen partition başarısına bağlıdır.

Algorithm

```
1  procedure quick_sort(A, p, r):  
2      if p < r:  
3          q = partition(A, p, r)  
4          quick_sort(A, p, q - 1)  
5          quick_sort(A, q + 1, r)
```

Heap Sort Pseudocode İncelemesi

- Dizi önce Max-Heap formatına sokulur (build_max_heap).
- En büyük eleman (kök) dizinin sonuyla swap edilir.
- Heap boyutu azaltılarak kök tekrar heapify yapılır.
- $n-1$ adım sonunda dizi tamamen sıralanmış olur.

Algorithm

```
1  procedure heap_sort(A):  
2      build_max_heap(A)  
3      for i = length(A) downto 2:  
4          swap(A[1], A[i])  
5          heap_size = heap_size - 1  
6          max_heapify(A, 1)
```

Hibrit Algoritmalar: Timsort ve Introsort

- Modern diller saf algoritmalar yerine hibritler kullanır.
- Timsort: Merge Sort ve Insertion Sort birleşimidir.
- Introsort: Quick, Heap ve Insertion Sort karmasıdır.
- Verinin karakteristiğine göre algoritma anlık değiştirilir.

Sıralama Algoritmaları Seçim Matrisi

- Kararlılık lazımsa Merge, ham hız lazımsa Quick seçilir.
- Bellek kısıtlıysa Heap Sort en güvenilir alternatiftir.
- Küçük dizilerde ($n < 15$) her zaman Insertion tercih edilir.
- Seçim matrisi, mühendislikte doğru aracı bulma rehberidir.

Hafta 3 Özeti ve Gelecek Haftaya Bakış

- Logaritmik algoritmalarla $n \log n$ duvarına ulaştık.
- Karşılaştırma tabanlı sistemlerin limitlerini kavradık.
- Haftaya: Karşılaştırma yapmadan lineer ($O(n)$) sıralama.
- Counting, Radix ve Bucket Sort ile sınırları zorlayacağız.

Soru 1 (Quick Sort Analizi)

Quick Sort algoritmasının "en kötü durum" (worst-case) senaryosunu açıklayınız ve asimptotik karmaşıklığını belirtiniz.

Çözüm Adımları

- **Dengesiz Bölünme:** En kötü durum, pivot seçiminin diziyi her seferinde çok dengesiz (örneğin $n - 1$ ve 0) iki parçaya bölmesiyle oluşur.
- **Tetikleyici Senaryo:** Bu durum genellikle **zaten sıralı** bir dizide, pivot olarak her zaman en uç (en küçük veya en büyük) elemanın seçilmesiyle gerçekleşir.
- **Rekürans Bağıntısı:** İşlem maliyeti $T(n) = T(n - 1) + T(0) + O(n)$ şekline dönüşür.
- **Matematiksel Toplam:** Bu, aritmetik bir dizi toplamına $(n + (n - 1) + \dots + 1)$ yol açar.
- **Sonuç:** Asimptotik olarak en yüksek terim n^2 olduğundan karmaşıklık $O(n^2)$ (Karesel Zaman) olur.

Learning Outcome: Quick Sort pivot analizi ve denge faktörü

Soru 2 (Merge Sort ve Kararlılık)

Merge Sort algoritmasının "kararlılık" (stability) özelliği neden önemlidir ve bu özellik algoritmanın hangi aşamasında korunur?

Çözüm Adımları

- **Kararlılık Tanımı:** Sıralama işlemi bittiğinde, aynı değere sahip elemanların birbirlerine göre olan orijinal (girdi) sıralarının değişmemesidir.
- **Kritik Aşama:** Kararlılık, parçalanmış dizilerin birleştirildiği `merge()` fonksiyonu sırasında korunur.
- **Koşul Mekanizması:** Birleştirme sırasında eğer sol dizideki eleman sağdakine eşitse ($A[i] \leq B[j]$), **sol dizideki elemana öncelik verilir.**

- **Önemli Fark:** Eğer sadece $<$ operatörü kullanılsaydı, eşit elemanların sırası değişebilir ve algoritma kararsız (unstable) hale gelirdi.
- **Kullanım Alanı:** Bu özellik, verilerin birden fazla anahtara göre (örneğin önce isme, sonra tarihe göre) sıralandığı karmaşık veri yapılarında tutarlılık için kritiktir.

Learning Outcome: Algoritma kararlılığı ve merge mantığı

Soru 3 (Uygulamalı Heap Analizi)

Verilen 12 elemanlı rastgele diziyi **Max-Heap** yapısına dönüştüren **Build-Heap** sürecini adım adım açıklayınız:

Dizi: [12, 45, 8, 70, 31, 92, 55, 6, 22, 18, 40, 33]

Çözüm ve Max-Heapify Süreci

- **Adım 1 (Başlangıç Noktası):** 12 elemanlı bir ağaçta son yaprak olmayan düğüm $n/2$ formülüyle 6. indekstir (92). İşlem buradan başlayarak köke doğru ($i = 6 \rightarrow 1$) ilerler.

Build-Heap: Tüm Adımların Detayı

İndeksler: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
Değerler: [12, 45, 8, 70, 31, 92, 55, 6, 22, 18, 40, 33]

****Adım 1.a (i=6):**** 6. indeksteki ****92**** incelenir. Tek çocuğu olan 12. indeksteki ****33**** ile karşılaştırılır. 92 > 33 olduğu için değişim yok.

****Adım 1.b (i=5):**** 5. indeksteki ****31**** incelenir. Çocukları 10. (18) ve 11. (40) indekslerdir. ****40**** daha büyük olduğu için 31 ile 40 yer değiştirir.

****Adım 1.c (i=4):**** 4. indeksteki ****70**** incelenir. Çocukları 8. (6) ve 9. (22) indekslerdir. 70 zaten en büyük, değişim yok.

****Adım 1.d (i=3):**** 3. indeksteki ****8**** incelenir. Çocukları 6. (92) ve 7. (55) indekslerdir. ****92**** ile 8 yer değiştirir. (8 aşağı süzülür).

****Adım 1.e (i=2):**** 2. indeksteki ****45**** incelenir. Çocukları 4. (70) ve 5. (yeni 40) indekslerdir. ****70**** ile 45 yer değiştirir. (45 aşağı süzülür).

****Adım 1.f (i=1 - KÖK):**** 1. indeksteki ****12**** incelenir. Çocukları 2. (70) ve 3. (92) arasından en büyüğü ****92****'dir. 12 ile 92 yer değiştirir. 12 aşağı süzölmeye devam ederek uygun yerine yerleşir.

Final Max-Heap: [92, 70, 55, 22, 40, 33, 8, 6, 12, 18, 31, 45] (Örnektir)

- **Adım 2 (Alt Seviye Kontrolü):** 92, 55 ve 33 değerleri incelenir. 92 zaten en büyük olduğu için bu alt ağaçta değişiklik yapılmaz. Ardından 31, 40 ve 18 incelenir; 40 yukarı çıkar.
- **Adım 3 (Zincirleme Etki - Ripple Effect):** 70 değeri, çocukları olan 6 ve 22 ile karşılaştırılır. 70 büyük olduğu için yerinde kalır.
- **Adım 4 (Yukarı Tırmanış):** Köke yaklaştıkça 45 ve 12 değerleri için **Max-Heapify** çağrılır. 45'in çocukları (70 ve 40) arasında 70 daha büyük olduğu için 45 ile 70 yer değiştirir.
- **Adım 5 (Final):** En son kökteki 12 değeri aşağı doğru süzülür (sift-down). 12, çocukları olan 70 ve 92 arasından en büyüğü olan 92 ile takas edilir ve uygun boşluğa yerleşene kadar devam eder.
- **Sonuç:** İşlem sonunda kök her zaman dizinin en büyüğüdür.

$O(n)$ maliyetli Build-Heap tamamlanmıştır.

Learning Outcome: Max-Heap yapısının bottom-up inşası

Heap-Sort: Sıralama Aşaması (Sorting Phase)

```
// HEAPSORT(A)

BUILD-MAX-HEAP(A) // (Az önce tamamladığımız kısım)

for i = A.length down to 2:

    exchange A[1] with A[i] // En büyüğü sona at

    A.heap-size = A.heap-size - 1

    MAX-HEAPIFY(A, 1) // Yeni kökü aşağı süz
```


1. Adım: En Büyüğü Sabitleme

Build-Heap sonrası kökteki 92 değerini, dizinin sonundaki (12. indeks) 45 ile takas ediyoruz. 92 artık "sıralanmış" kabul edilir ve heap dışı kalır.

2. Adım: Max-Heapify (Yeniden Yapılandırma)

Yeni kök olan 45, Max-Heap kuralını bozabilir. 45'in çocukları (70 ve 55) incelenir. 70 daha büyük olduğu için 45 ile 70 yer değiştirir. Bu işlem 45 uygun yere inene kadar sürer.

3. Adım: Döngü Devamı

Şimdi kökteki yeni en büyük (70), dizinin sondan ikinci (11. indeks) elemanı ile takas edilir. Dizi boyutu hayali olarak 10'a düşer ve tekrar kök için **Max-Heapify** çağrılır.

Karmaşıklık: $n-1$ kez Max-Heapify çağrıldığı için toplam süre $O(n \log n)$ olur.

Soru 4 (İleri Seviye Analiz)

Heap Sort algoritmasında "Build-Heap" aşamasının neden **$O(n)$** zamanda bittiğini matematiksel olarak nasıl açıklarsınız?

İspat Adımları

- **Yapısal Gözlem:** Build-Heap işlemi yapraklardan yukarı doğru (bottom-up) çalışır. Yapraklar zaten birer "heap" olduğu için işlem $n/2$ yüksekliğinden başlar.
- **Düğüm Dağılımı:** Bir tam ikili ağaçta, h yüksekliğindeki bir düğümün seviyesindeki maksimum düğüm sayısı yaklaşık $\lceil n/2^{h+1} \rceil$ kadardır.
- **Maliyet Formülü:** Toplam iş, her seviyedeki düğüm sayısı ile o düğümün inebileceği maksimum derinliğin çarpımıdır: $\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} O(h)$

- **Geometrik Seri:** Sabitleri dışarı aldığımızda ifade $n \cdot \sum_{h=0}^{\infty} \frac{h}{2^h}$ halini alır. Bu sonsuz geometrik seri 2 değerine yakınsar.
- **Sonuç:** Toplam maliyet $n \cdot 2$ mertebesinde. Sabit çarpan ihmal edildiğinde üst sınır $O(n)$ olarak ispatlanır.

Learning Outcome: Heap inşa karmaşıklığı ispatı